

Project Executable Files

Date	05 July 2026
Team ID	SWTID-2026-2662
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	3 Marks

This section lists all executable and deployable files for FinRelief AI, organized so the evaluator can set up and run the solution locally using the source code and instructions provided in the project repository.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted	Notes
1	Complete source code (all files and folders)	Yes	37 files across app/ (FastAPI backend) and streamlit_app/ (frontend), included in the project report Appendix.
2	README / Setup Guide (instructions to run the project)	Yes	"How to Run" instructions documented (see Step 4 below).
3	requirements.txt / package.json / dependency file	Yes	requirements.txt with 11 pinned package versions; runtime.txt specifies Python 3.11.0.
4	Database schema / seed files (if applicable)	NA	No separate seed file — schema is defined via SQLAlchemy ORM models (app/models/) and created automatically on first run.
5	Environment configuration file (.env.example or similar)	Yes	.env template provided with GOOGLE_API_KEY, DATABASE_URL, SECRET_KEY, and API_BASE_URL.
6	Deployed application URL (if hosted)	No	Not yet deployed to a live URL. The application currently runs locally (see Known Issues). Cloud deployment is listed as a pending item in the project's Future Scope.
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA	Not applicable — FinRelief AI is a web application (Streamlit + FastAPI), not a mobile or desktop app.
8	Dockerfile / Containerization config (if applicable)	No	Not yet created. Docker-based deployment is proposed in the Technology Stack section as a future step but not implemented in the current release.
9	Test files and test results	Yes	Functional and performance test results documented (Apache JMeter reports, functional test cases FT-01–FT-04); see the project's Testing sections.
10	Demo video or walkthrough (if required)	In Progress	Demo video script prepared; recording pending at time of this submission.

Step 2: File / Folder Structure

The actual folder structure of the FinRelief AI codebase, as implemented:

```
FinRelief-AI/
├── app/                                     (FastAPI backend)
│   ├── __init__.py
│   ├── main.py                             - App entry point, CORS, router registration
│   ├── database.py                         - SQLAlchemy engine & session config
│   ├── auth.py                             - JWT token creation & validation
│   ├── models/                             - SQLAlchemy ORM models
│   │   ├── user.py, loan.py, financial_profile.py
│   │   └── settlement.py, ai_models.py
│   ├── schemas/                             - Pydantic request/response schemas
│   │   └── user.py, loan.py
│   ├── routers/                             - API route handlers
│   │   ├── auth.py, profile.py, loans.py, dashboard.py, ai.py
│   └── services/                             - Business logic
│       ├── financial_engine.py, settlement_engine.py, ai_engine.py
├── streamlit_app/                           (Frontend)
│   ├── app.py                             - Main entry point, session/navigation
│   ├── api_client.py                       - HTTP client to backend API
│   ├── components.py                       - Reusable UI component helpers
│   ├── styles.css                           - Custom dark-theme styling
│   └── pages/                               - Individual Streamlit pages
│       ├── p01_dashboard.py, p02_financial_health.py
│       ├── p03_settlement_predictor.py, p04_negotiation_email.py
│       └── p05_know_your_rights.py, p06_history.py
├── requirements.txt                         - Pinned Python dependencies
├── runtime.txt                             - Python 3.11.0
└── .env                                    - Environment configuration template
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not yet deployed — runs locally at http://localhost:8501 (frontend) with the API at http://localhost:8000 .
Login Credentials (Demo)	No pre-seeded demo account exists; the evaluator should register a new account via the app's Register tab, then log in with those credentials.
Platform / Hosting Provider	Not yet hosted. Docker with Render (or AWS EC2 / Elastic Beanstalk) is the proposed hosting approach documented in the Technology Stack section — not yet implemented.
Repository Link	https://github.com/nakkahemasaisree-cmd/AI-powered-debt-relief-and-financial-recovery-platform
Demo Video Link	https://drive.google.com/file/d/1AoQy2YSE4mAHeExVLr-KgtKG2Yc36TBm/view?usp=sharing

Step 4: Run Instructions

Step-by-step instructions for the evaluator to run the project locally:

Pre-requisites

Python 3.11.x (project's runtime.txt specifies 3.11.0). A Google Gemini API key is optional — if left blank in .env, the AI features automatically use a rule-based fallback instead of failing.

```
Step 1: Clone the repository
git clone <repository-url>
cd FinRelief-AI

Step 2 (Terminal 1 – Backend):
python -m venv venv
venv\Scripts\activate      (Windows)
source venv/bin/activate   (Mac/Linux)
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

Step 3 (Terminal 2 – Frontend):
venv\Scripts\activate      (or source venv/bin/activate)
streamlit run streamlit_app/app.py

Step 4: Configure environment
Copy .env and fill in GOOGLE_API_KEY (optional), DATABASE_URL,
SECRET_KEY, and API_BASE_URL before starting the backend.

Step 5: Open the application
http://localhost:8501
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Initial environment setup can fail if the installed Python version does not match the pinned version (3.11.0) in runtime.txt.	Install Python 3.11 and recreate the virtual environment before installing dependencies. Documented in the project's UAT report.
2	Minor code errors were encountered by some team members on first run after cloning the repository.	Resolved using the VS Code AI coding assistant; no issues remain in the current codebase.
3	SQLite is used as the database, which is suitable for development/small-scale use but not for high-concurrency production deployment.	Documented in the Scalability & Future Plan — migration to PostgreSQL/MySQL is planned for Phase 3 of the roadmap.
4	The application relies on manually entered financial data rather than real-time bank/credit bureau integration.	Planned for Phase 2 of the Future Scope roadmap (real-time banking API integration).
5	Application is not yet containerized or deployed to a live URL.	Docker and cloud hosting (Render/AWS) are proposed but not yet implemented; currently run locally only.